

Best Model Selection to Predict Risk of Heart Disease

First, we need to import all the required libraries.

```
In [3]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.svm import SVC
from sklearn.metrics import ConfusionMatrixDisplay, RocCurveDisplay, accuracy_score, precision_score, recall_score, f1_score
```

Importing the dataset and have a look at the first five rows.

```
In [4]: heart_data = pd.read_csv('heart_disease_risk_dataset_earlymed.csv')
heart_data.head()
```

```
Out[4]:
```

	Chest_Pain	Shortness_of_Breath	Fatigue	Palpitations	Dizziness	Swelling	Pain_Arms_Jaw_Back	Cold_Sweats_Nausea	High_Blood_Pressure
0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0
1	0.0	1.0	0.0	1.0	0.0	0.0	0.0	0.0	1
2	1.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	1
3	1.0	1.0	0.0	1.0	0.0	0.0	1.0	1.0	1
4	0.0	0.0	1.0	0.0	1.0	0.0	0.0	0.0	0



Also, check the last five rows to get an idea about the data.

```
In [5]: heart_data.tail()
```

```
Out[5]:
```

	Chest_Pain	Shortness_of_Breath	Fatigue	Palpitations	Dizziness	Swelling	Pain_Arms_Jaw_Back	Cold_Sweats_Nausea	H
69995	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	
69996	1.0	1.0	1.0	0.0	1.0	1.0	1.0	1.0	
69997	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	
69998	1.0	1.0	0.0	1.0	1.0	0.0	1.0	1.0	
69999	1.0	1.0	1.0	1.0	1.0	1.0	1.0	0.0	



By looking at the first and last five rows, we can see that this dataset contains all numeric values.

This dataset has 70,000 rows and 19 columns.

```
In [6]: heart_data.shape
```

```
Out[6]: (70000, 19)
```

View the summary of data columns and their data types.

```
In [5]: heart_data.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 70000 entries, 0 to 69999
Data columns (total 19 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Chest_Pain             70000 non-null  float64
1   Shortness_of_Breath    70000 non-null  float64
2   Fatigue                70000 non-null  float64
3   Palpitations           70000 non-null  float64
4   Dizziness              70000 non-null  float64
5   Swelling               70000 non-null  float64
6   Pain_Arms_Jaw_Back     70000 non-null  float64
7   Cold_Sweats_Nausea     70000 non-null  float64
8   High_BP                70000 non-null  float64
9   High_Cholesterol        70000 non-null  float64
10  Diabetes                70000 non-null  float64
11  Smoking                 70000 non-null  float64
12  Obesity                 70000 non-null  float64
13  Sedentary_Lifestyle     70000 non-null  float64
14  Family_History          70000 non-null  float64
15  Chronic_Stress          70000 non-null  float64
16  Gender                  70000 non-null  float64
17  Age                     70000 non-null  float64
18  Heart_Risk              70000 non-null  float64
dtypes: float64(19)
memory usage: 10.1 MB

```

Check whether there are any missing values in each column. The following shows that there are none.

```
In [6]: heart_data.isnull().sum()
```

```
Out[6]: Chest_Pain      0
        Shortness_of_Breath  0
        Fatigue        0
        Palpitations    0
        Dizziness       0
        Swelling        0
        Pain_Arms_Jaw_Back  0
        Cold_Sweats_Nausea  0
        High_BP        0
        High_Cholesterol  0
        Diabetes        0
        Smoking        0
        Obesity         0
        Sedentary_Lifestyle  0
        Family_History  0
        Chronic_Stress  0
        Gender         0
        Age            0
        Heart_Risk      0
        dtype: int64
```

Check whether there are any duplicates available in the dataset. The results shows that there are 6,245 duplicates, which were removed.

```
In [7]: heart_data.duplicated().sum()
```

```
Out[7]: np.int64(6245)
```

```
In [8]: heart_data = heart_data.drop_duplicates()
```

```
In [9]: heart_data.duplicated().sum()
```

```
Out[9]: np.int64(0)
```

Since this is a binary classification problem, there are only two categories - whether some one has a risk of heart disease or not. The following count plot shows the number of positive(1 - at a risk) and negative (0 - not at a risk).

This graph shows there is very small imbalance in the dataset. Since it is insignificant, I decided to proceed without addressing the imbalance using oversampling or undersampling techniques. Instead used hyperparameters such as 'class_weight = 'balanced''

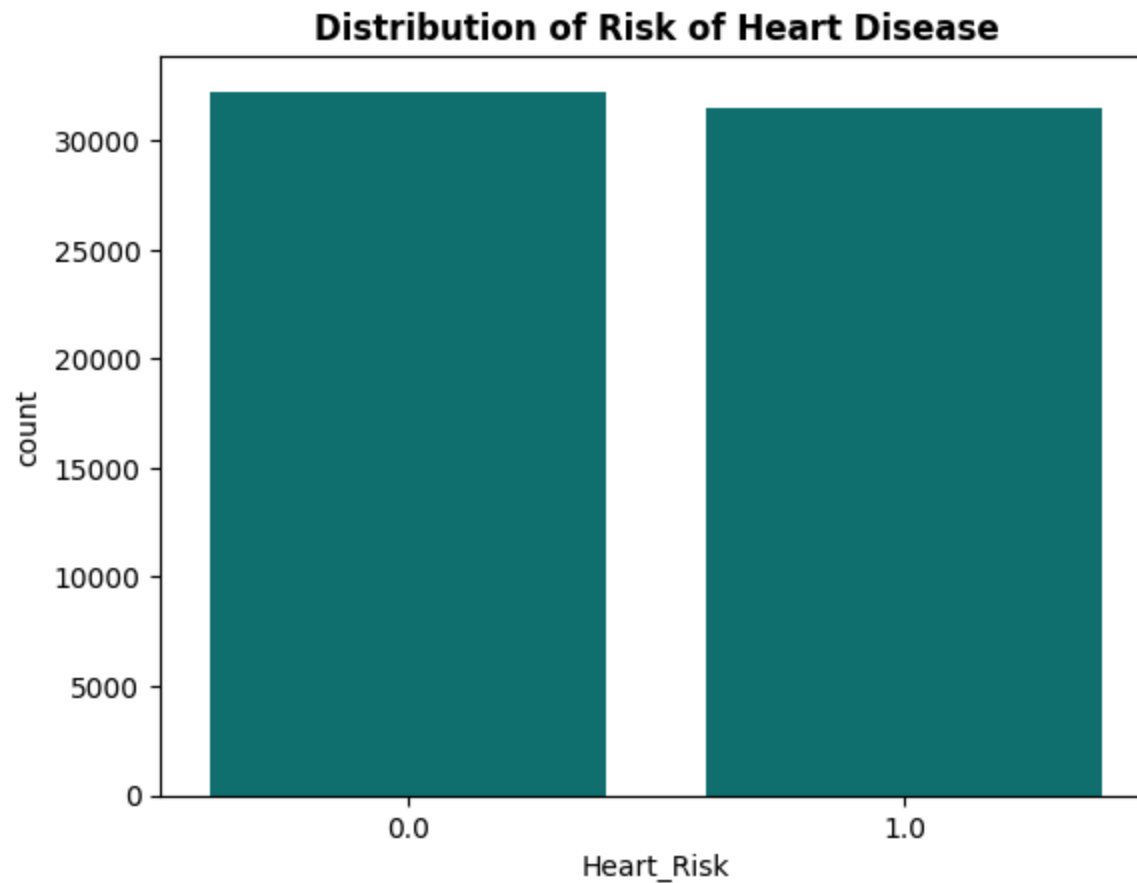
option.

```
In [10]: heart_data['Heart_Risk'].value_counts()
```

```
Out[10]: Heart_Risk  
0.0    32268  
1.0    31487  
Name: count, dtype: int64
```

```
In [11]: sns.countplot(data = heart_data, x = 'Heart_Risk', color='teal')  
plt.title("Distribution of Risk of Heart Disease", fontweight = 'bold')
```

```
Out[11]: Text(0.5, 1.0, 'Distribution of Risk of Heart Disease')
```



The following method shows all the available column names.

```
In [12]: heart_data.columns
```

```
Out[12]: Index(['Chest_Pain', 'Shortness_of_Breath', 'Fatigue', 'Palpitations',  
              'Dizziness', 'Swelling', 'Pain_Arms_Jaw_Back', 'Cold_Sweats_Nausea',  
              'High_BP', 'High_Cholesterol', 'Diabetes', 'Smoking', 'Obesity',  
              'Sedentary_Lifestyle', 'Family_History', 'Chronic_Stress', 'Gender',  
              'Age', 'Heart_Risk'],  
             dtype='object')
```

Below graph below shows the impact of each feature on the risk of having heart disease. The impact of gender is highlighted.

```
In [7]: plt.figure(figsize = (17,20))  
plt.suptitle("Risk of Heart Desease by Each Feature Variable", fontsize = 25, fontweight = 'bold')  
  
plt.subplot(5,4,1)  
sns.countplot(data = heart_data, x = 'Chest_Pain', color='cornflowerblue')  
plt.yticks(fontsize = 6)  
  
plt.subplot(5,4,2)  
sns.countplot(data = heart_data, x = 'Shortness_of_Breath', color= 'cadetblue')  
plt.yticks(fontsize = 6)  
  
plt.subplot(5,4,3)  
sns.countplot(data = heart_data, x = 'Fatigue', color='darkcyan')  
plt.yticks(fontsize = 6)  
  
plt.subplot(5,4,4)  
sns.countplot(data = heart_data, x = 'Palpitations', color='dodgerblue')  
plt.yticks(fontsize = 6)  
  
plt.subplot(5,4,5)  
sns.countplot(data = heart_data, x = 'Dizziness', color='cornflowerblue')  
plt.yticks(fontsize = 6)  
  
plt.subplot(5,4,6)  
sns.countplot(data = heart_data, x = 'Swelling', color= 'cadetblue')  
plt.yticks(fontsize = 6)  
  
plt.subplot(5,4,7)
```

```
sns.countplot(data = heart_data, x = 'Pain_Arms_Jaw_Back', color='darkcyan')
plt.yticks(fontsize = 6)

plt.subplot(5,4,8)
sns.countplot(data = heart_data, x = 'Cold_Sweats_Nausea', color='dodgerblue')
plt.yticks(fontsize = 6)

plt.subplot(5,4,9)
sns.countplot(data = heart_data, x = 'High_BP', color='cornflowerblue')
plt.yticks(fontsize = 6)

plt.subplot(5,4,10)
sns.countplot(data = heart_data, x = 'High_Cholesterol', color= 'cadetblue')
plt.yticks(fontsize = 6)

plt.subplot(5,4,11)
sns.countplot(data = heart_data, x = 'Diabetes', color='darkcyan')
plt.yticks(fontsize = 6)

plt.subplot(5,4,12)
sns.countplot(data = heart_data, x = 'Smoking', color='dodgerblue')
plt.yticks(fontsize = 6)

plt.subplot(5,4,13)
sns.countplot(data = heart_data, x = 'Obesity', color='cornflowerblue')
plt.yticks(fontsize = 6)

plt.subplot(5,4,14)
sns.countplot(data = heart_data, x = 'Sedentary_Lifestyle', color= 'cadetblue')
plt.yticks(fontsize = 6)

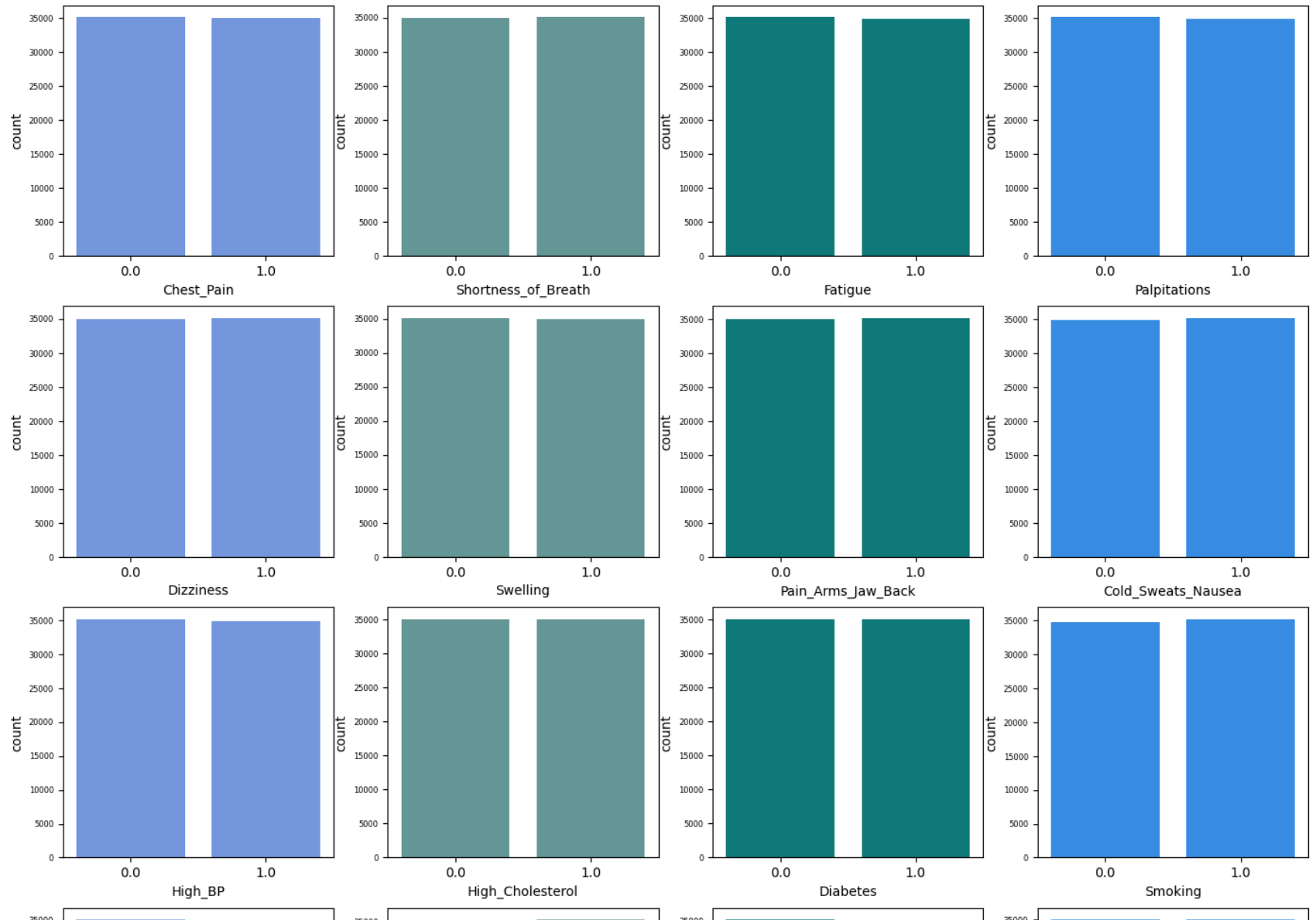
plt.subplot(5,4,15)
sns.countplot(data = heart_data, x = 'Family_History', color='darkcyan')
plt.yticks(fontsize = 6)

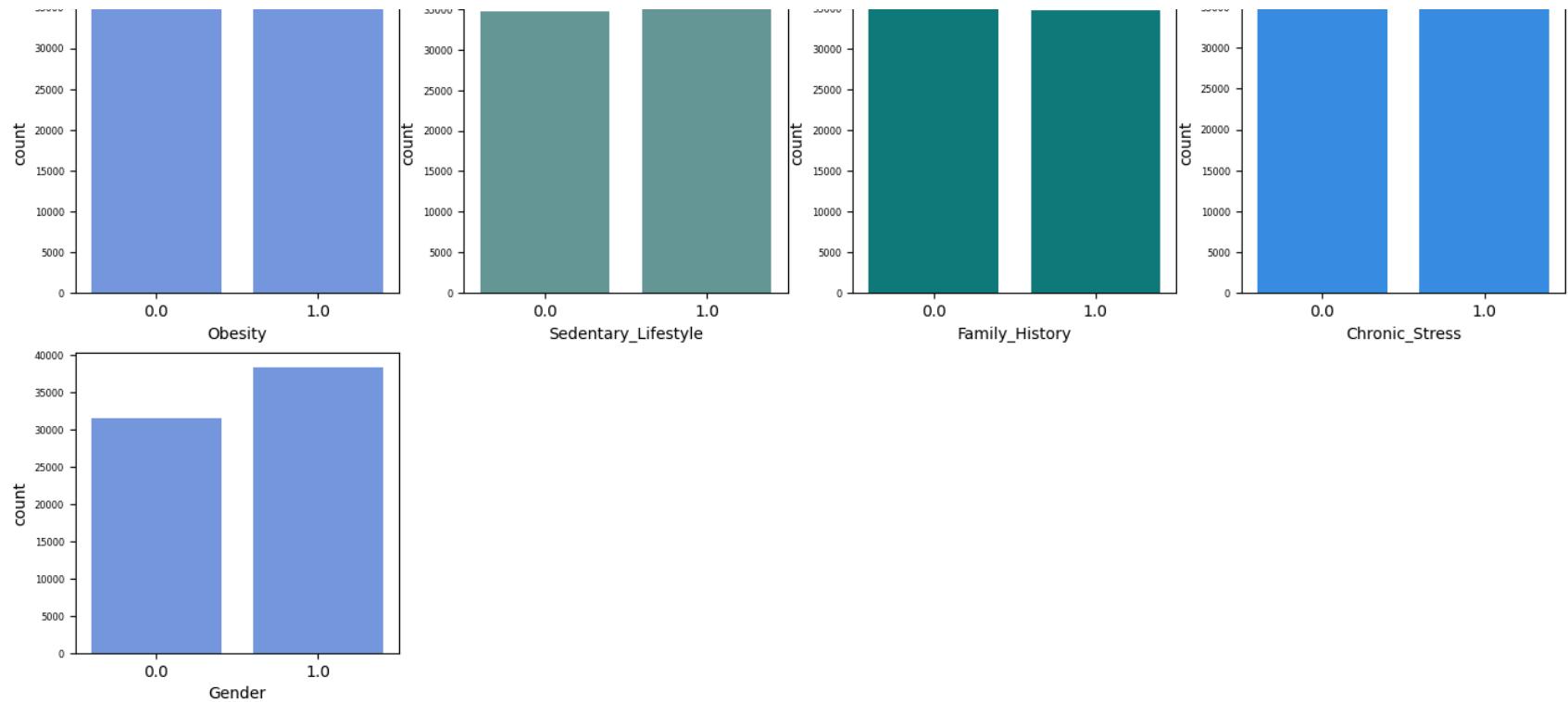
plt.subplot(5,4,16)
sns.countplot(data = heart_data, x = 'Chronic_Stress', color='dodgerblue')
plt.yticks(fontsize = 6)

plt.subplot(5,4,17)
sns.countplot(data = heart_data, x = 'Gender', color='cornflowerblue')
plt.yticks(fontsize = 6)
```

```
Out[7]: (array([ 0., 5000., 10000., 15000., 20000., 25000., 30000., 35000.,
                40000., 45000.]),
         [Text(0, 0.0, '0'),
          Text(0, 5000.0, '5000'),
          Text(0, 10000.0, '10000'),
          Text(0, 15000.0, '15000'),
          Text(0, 20000.0, '20000'),
          Text(0, 25000.0, '25000'),
          Text(0, 30000.0, '30000'),
          Text(0, 35000.0, '35000'),
          Text(0, 40000.0, '40000'),
          Text(0, 45000.0, '45000')])
```

Risk of Heart Disease by Each Feature Variable



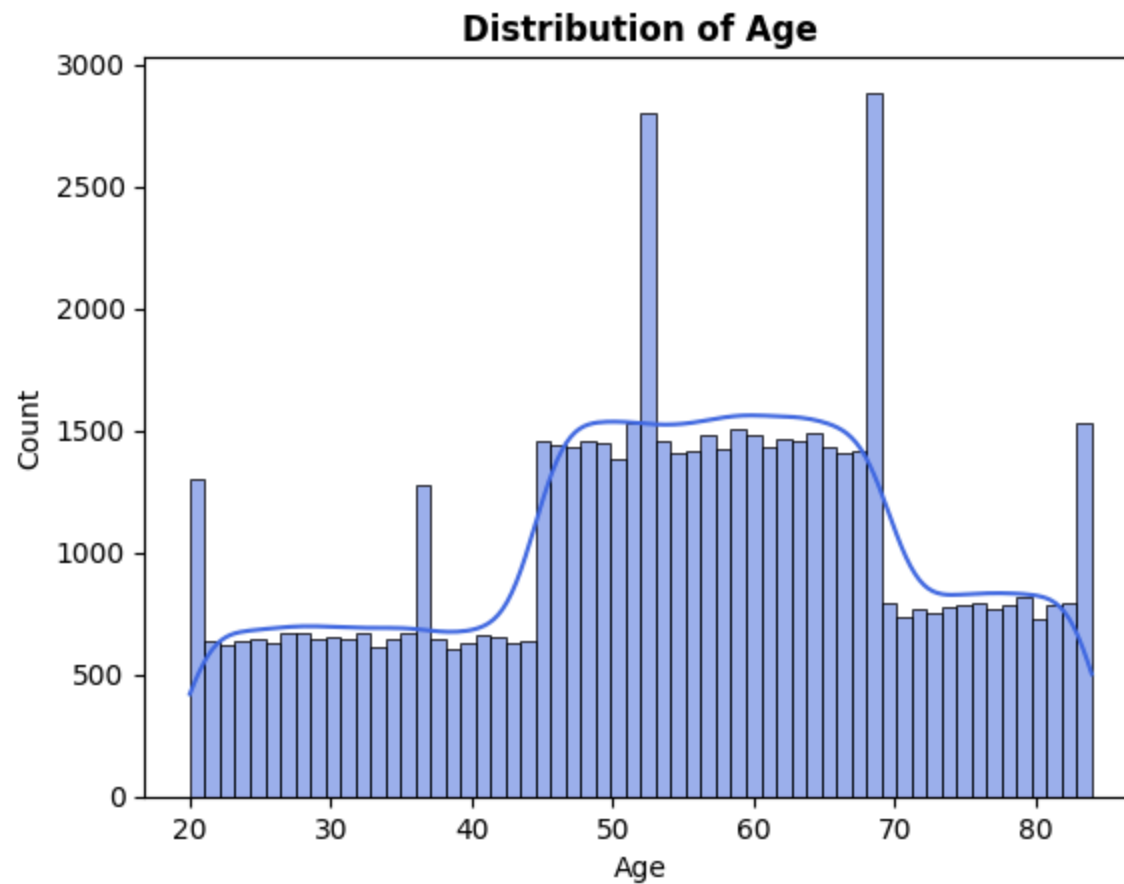


In this dataset Age is only feature/variable that has range of values. All other are binary inputs (1 - Yes or 0 - No)

Therefore, using a histogram, I checked whether it has any outliers and it is not normally distributed. Therefore, the 'Age' feature needs to be normalized.

```
In [16]: sns.histplot(data = heart_data, x = 'Age', kde = True, color='royalblue', bins = 60)
plt.title('Distribution of Age', fontweight = 'bold')
```

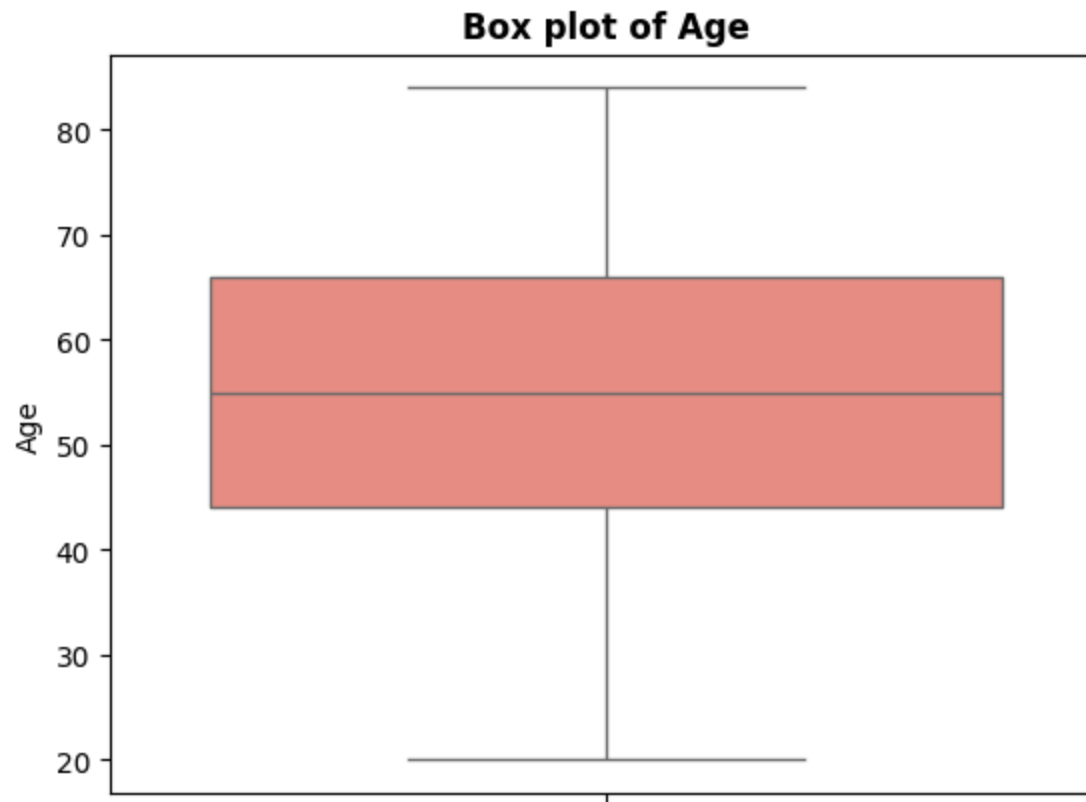
```
Out[16]: Text(0.5, 1.0, 'Distribution of Age')
```



Using a boxplot, it was further verified whether the 'Age' feature has outliers. By looking at the following box plot, we can see that there are no outliers present.

```
In [14]: sns.boxplot(data = heart_data, y = 'Age', color='salmon')
plt.title('Box plot of Age', fontweight = 'bold')
```

```
Out[14]: Text(0.5, 1.0, 'Box plot of Age')
```

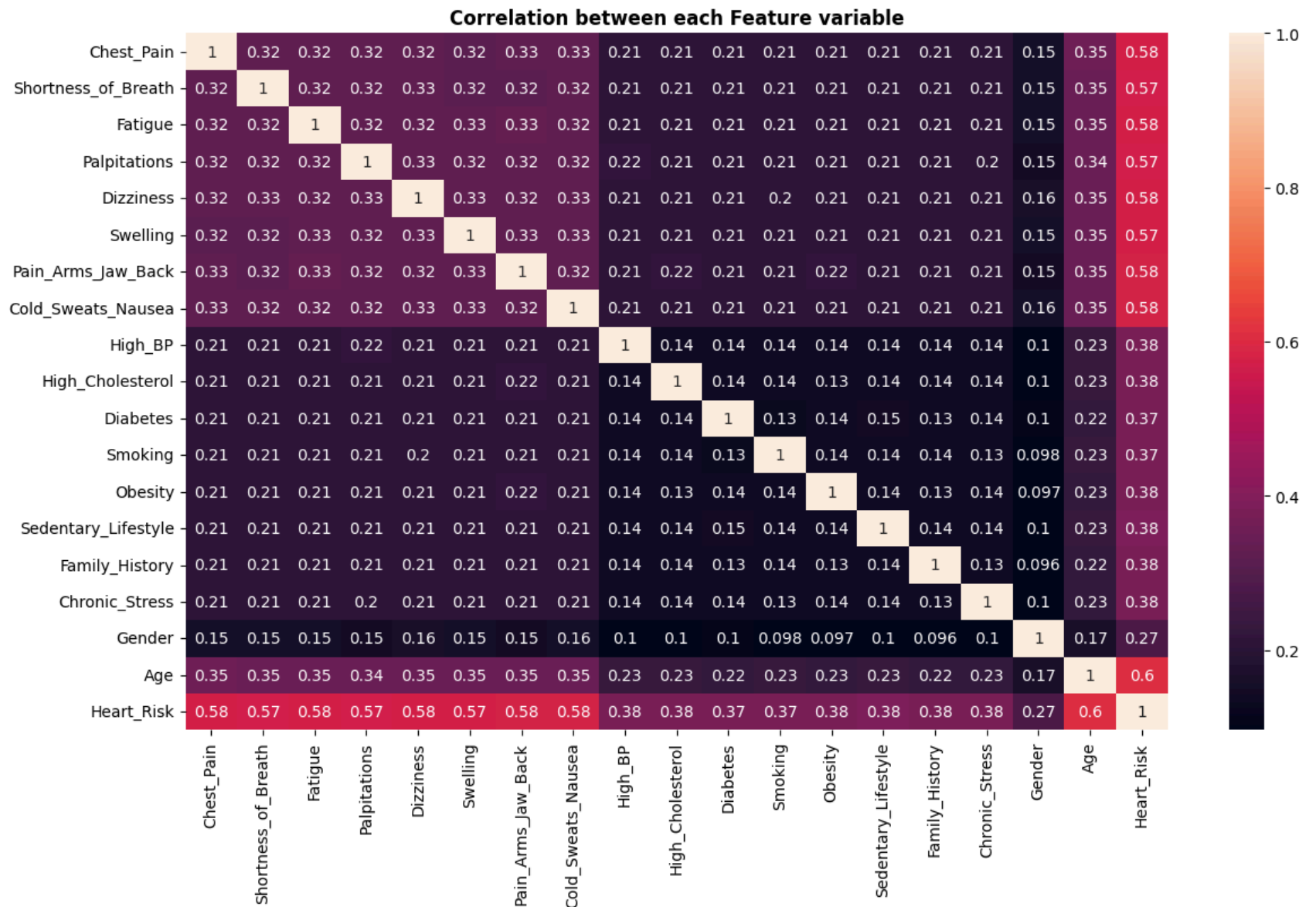


To get an idea about the correlations between features, a heatmap was used. By observing the heatmap, we can say that the with higher correlation to Heart_Risk are: Chest pain, Shortness of breath, Fatigue, Palpitations, Dizziness, Swelling, Pain in arms/jaw/back, Cold sweats, Nausea and Age. Among these features, except for Age, the rest are recorded after the person had a heart attack. This represents data leakage. So, Those features need to be removed.

```
In [15]: correations = heart_data.corr()

plt.figure(figsize = (14,8))
sns.heatmap(correations, annot = True)
plt.title('Correlation between each Feature variable', fontweight = 'bold')
```

```
Out[15]: Text(0.5, 1.0, 'Correlation between each Feature variable')
```



Make a copy of original dataset, and perform all changes on the copy from here onwards.

```
In [17]: heart_data_copy = heart_data.copy()
```

Numerical columns are put into an array. In this case 'Age' is only numerical feature.

```
In [19]: numerical_cols = ['Age']
```

Addressing data leakage by ensuring unnecessary columns are removed.

```
In [20]: data_leakage_cols = ['Chest_Pain', 'Shortness_of_Breath', 'Fatigue', 'Palpitations', 'Dizziness', 'Swelling', 'Pain_Arms_Joints',  
                             'Cold_Sweats_Nausea']
```

```
In [21]: heart_data_copy_cleaned = heart_data_copy.drop(columns = data_leakage_cols)
```

```
In [22]: heart_data_copy_cleaned.columns
```

```
Out[22]: Index(['High_BP', 'High_Cholesterol', 'Diabetes', 'Smoking', 'Obesity',  
               'Sedentary_Lifestyle', 'Family_History', 'Chronic_Stress', 'Gender',  
               'Age', 'Heart_Risk'],  
              dtype='object')
```

Define X and y

```
In [23]: y = heart_data_copy_cleaned['Heart_Risk']  
X = heart_data_copy_cleaned.drop(columns = 'Heart_Risk')
```

```
In [24]: X.columns
```

```
Out[24]: Index(['High_BP', 'High_Cholesterol', 'Diabetes', 'Smoking', 'Obesity',  
               'Sedentary_Lifestyle', 'Family_History', 'Chronic_Stress', 'Gender',  
               'Age'],  
              dtype='object')
```

Split the data into training and testing sets.

```
In [25]: X_train, X_test, y_train, y_test = train_test_split(X,y, test_size = 0.2, random_state=12, stratify= y )
```

Normalizing the X data

```
In [26]: scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train[numerical_cols])
X_test_scaled = scaler.transform(X_test[numerical_cols])
```

Verify that the data has been normalized correctly. Normalized data should have a mean of zero and a standard deviation of one.

```
In [27]: round(np.mean(X_test_scaled),2)
```

```
Out[27]: np.float64(-0.01)
```

```
In [28]: round(np.std(X_test_scaled),2)
```

```
Out[28]: np.float64(1.0)
```

Defining all the machine learning algorithms expected to be used for this dataset.

```
In [29]: model_1 = LogisticRegression(random_state=12, class_weight='balanced')
model_2 = DecisionTreeClassifier(random_state=12, class_weight='balanced')
model_3 = RandomForestClassifier(random_state=12, class_weight='balanced')
model_4 = XGBClassifier()
model_5 = MLPClassifier(random_state=12, max_iter=500)
```

Using GridSearchCV to choose the best combination of parameters. After selecting best parameters, run each algorithm one by one and evaluate them separately. At the end, the ROC_Curve for all models together and choose the best model to make predictions for this dataset.

```
In [30]: params_1 = {
    'C': [0.01, 0.1, 1, 1.5, 2],
    'max_iter': [100, 150, 200, 250],
    'solver': ['lbfgs', 'liblinear', 'sag', 'saga']
}

params_2 = {
    'max_depth': [5, 10, 15, 20],
    'min_samples_split': [2, 3, 4]
}

params_3 = {
    'n_estimators': [100, 200, 250],
    'max_depth': [2, 5, 10, 15],
```

```

    'min_samples_split': [2,3,4]
}

params_4 = {
    'scale_pos_weight': [1,2,3],    #using when the dataset is imbalances
    'learning_rate': [0.001,0.01,0.1,0.3],
    'gamma': [0,1,2,3,4]

}

params_5 = {
    'hidden_layer_sizes': [(16,),(16,32)],
    'solver': ['adam','sgd','lbfgs'],
    'alpha': [0.001,0.01,0.1,0.5]
}

```

```

In [33]: grid_1 = GridSearchCV(model_1, params_1, cv = 5, scoring='f1')
grid_1.fit(X_train_scaled, y_train)

print(grid_1.best_params_)
print(grid_1.best_score_)

{'C': 0.1, 'max_iter': 100, 'solver': 'sag'}
0.7215579898715824

```

```

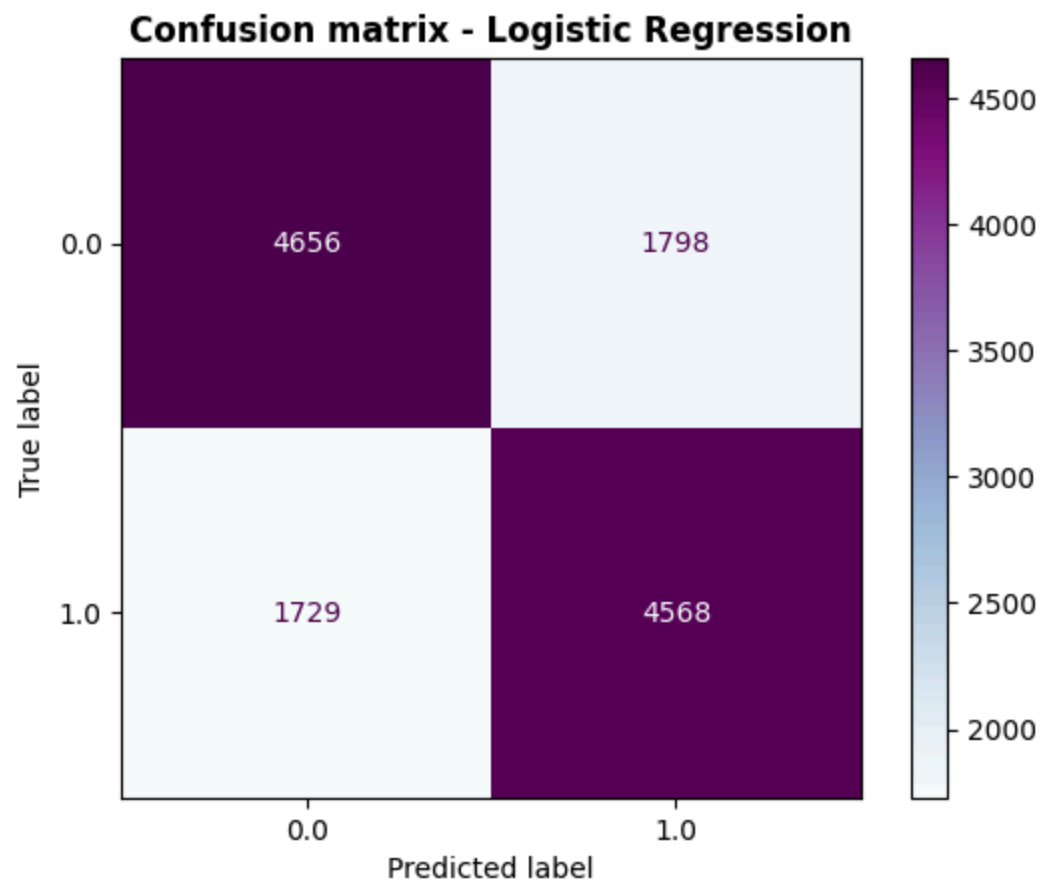
In [31]: model1 = LogisticRegression(random_state=12, class_weight='balanced',max_iter=100, C = 0.1, solver='sag')
model1.fit(X_train_scaled,y_train)
pred_y_1 = model1.predict(X_test_scaled)
ConfusionMatrixDisplay.from_predictions(y_test, pred_y_1, cmap = 'BuPu')
plt.title("Confusion matrix - Logistic Regression", fontweight = 'bold')

```

```

Out[31]: Text(0.5, 1.0, 'Confusion matrix - Logistic Regression')

```



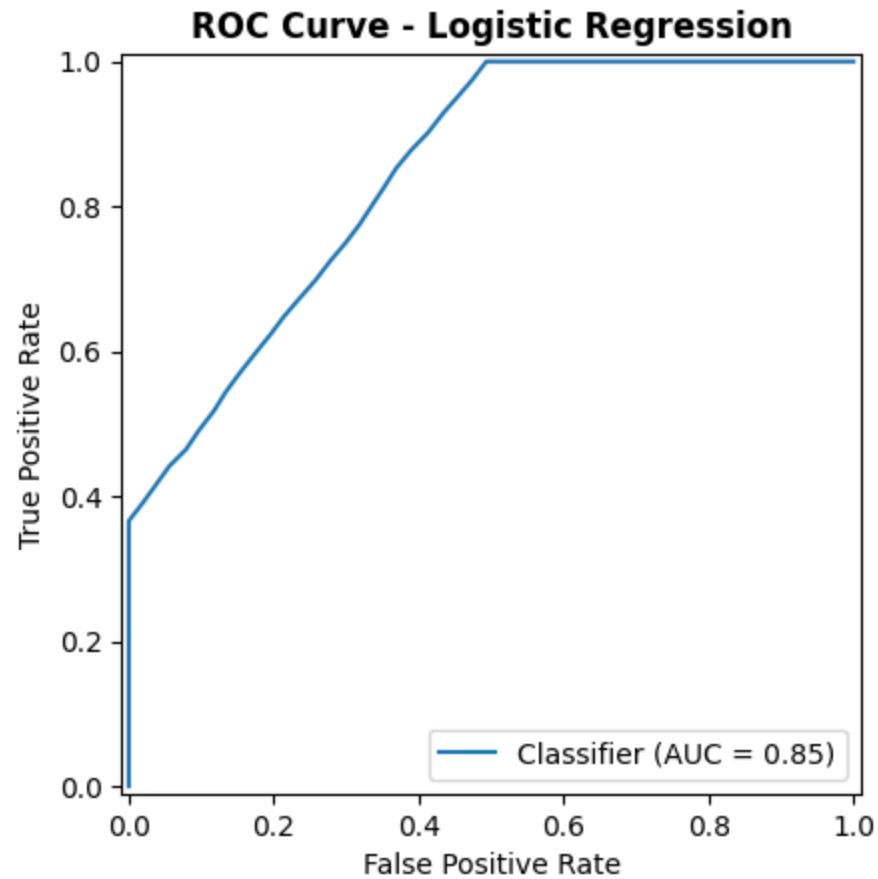
```
In [32]: print("Accuracy : ",round(accuracy_score(y_test, pred_y_1),2))
print("Precision : ",round(precision_score(y_test, pred_y_1),2))
print("Recall : ",round(recall_score(y_test, pred_y_1),2))
print("f1_Score : ",round(f1_score(y_test, pred_y_1),2))
```

```
Accuracy : 0.72
Precision : 0.72
Recall : 0.73
f1_Score : 0.72
```

```
In [40]: y_scores = model1.predict_proba(X_test_scaled)[: ,1]
RocCurveDisplay.from_predictions(y_test,y_scores)
plt.title("ROC Curve - Logistic Regression", fontweight = 'bold') # based on positive label 1
```

```
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
```

Out[40]: Text(0, 0.5, 'True Positive Rate')



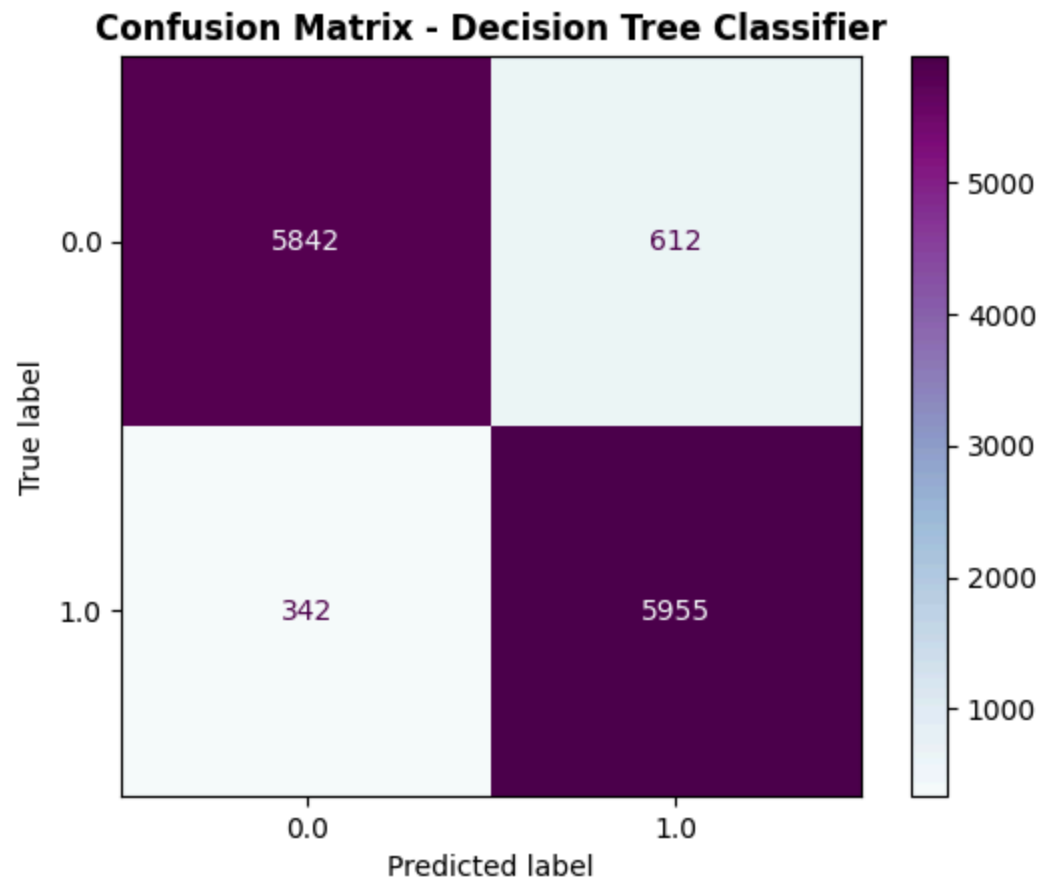
```
In [ ]: grid_2 = GridSearchCV(model_2, params_2, cv = 5, scoring = 'f1')
grid_2.fit(X_train, y_train)

print(grid_2.best_params_)
print(grid_2.best_score_)
```

```
{'max_depth': 10, 'min_samples_split': 2}
0.9253275494186555
```

```
In [34]: model2 = DecisionTreeClassifier(random_state=12, class_weight='balanced', max_depth=10, min_samples_split=2)
model2.fit(X_train, y_train)
pred_y_2 = model2.predict(X_test)
ConfusionMatrixDisplay.from_predictions(y_test, pred_y_2, cmap = 'BuPu')
plt.title("Confusion Matrix - Decision Tree Classifier", fontweight = 'bold')
```

```
Out[34]: Text(0.5, 1.0, 'Confusion Matrix - Decision Tree Classifier')
```

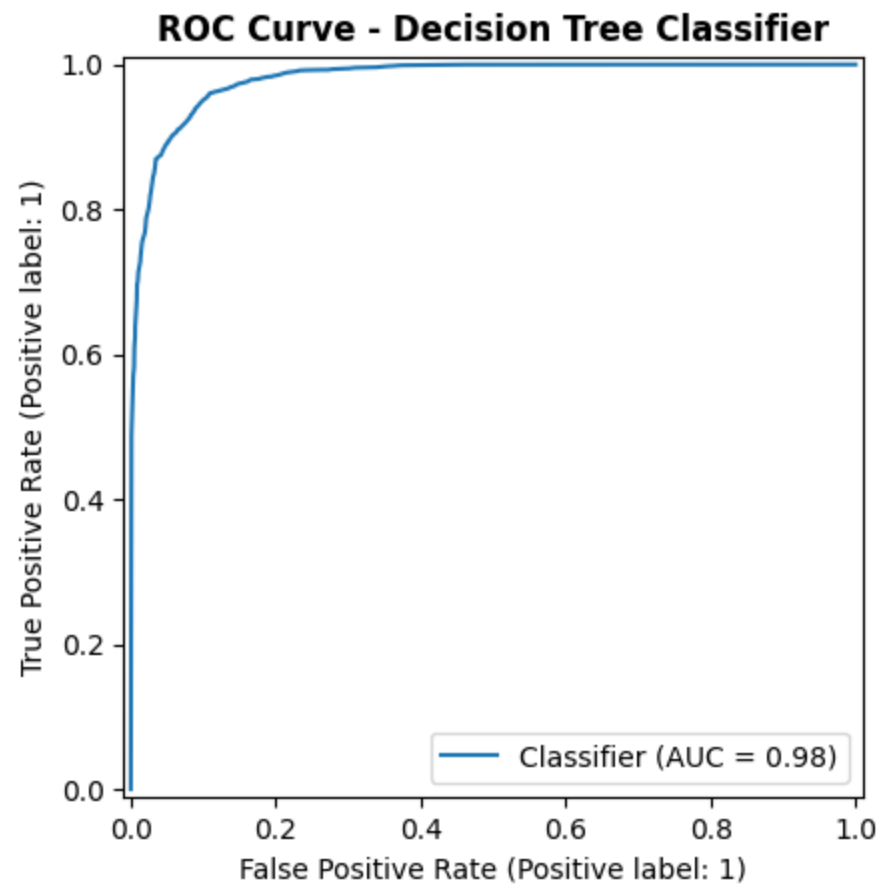


```
In [35]: print("Accuracy : ",round(accuracy_score(y_test, pred_y_2),2))
print("Precision : ",round(precision_score(y_test, pred_y_2),2))
print("Recall : ",round(recall_score(y_test, pred_y_2),2))
print("f1_Score : ",round(f1_score(y_test, pred_y_2),2))
```

```
Accuracy : 0.93  
Precision : 0.91  
Recall : 0.95  
f1_Score : 0.93
```

```
In [ ]: y_scores = model2.predict_proba(X_test)[: ,1]  
  
RocCurveDisplay.from_predictions(y_test, y_scores)  
plt.title("ROC Curve - Decision Tree Classifier", fontweight = 'bold')  
plt.xlabel("")
```

```
Out[ ]: Text(0.5, 1.0, 'ROC Curve - Decision Tree Classifier')
```



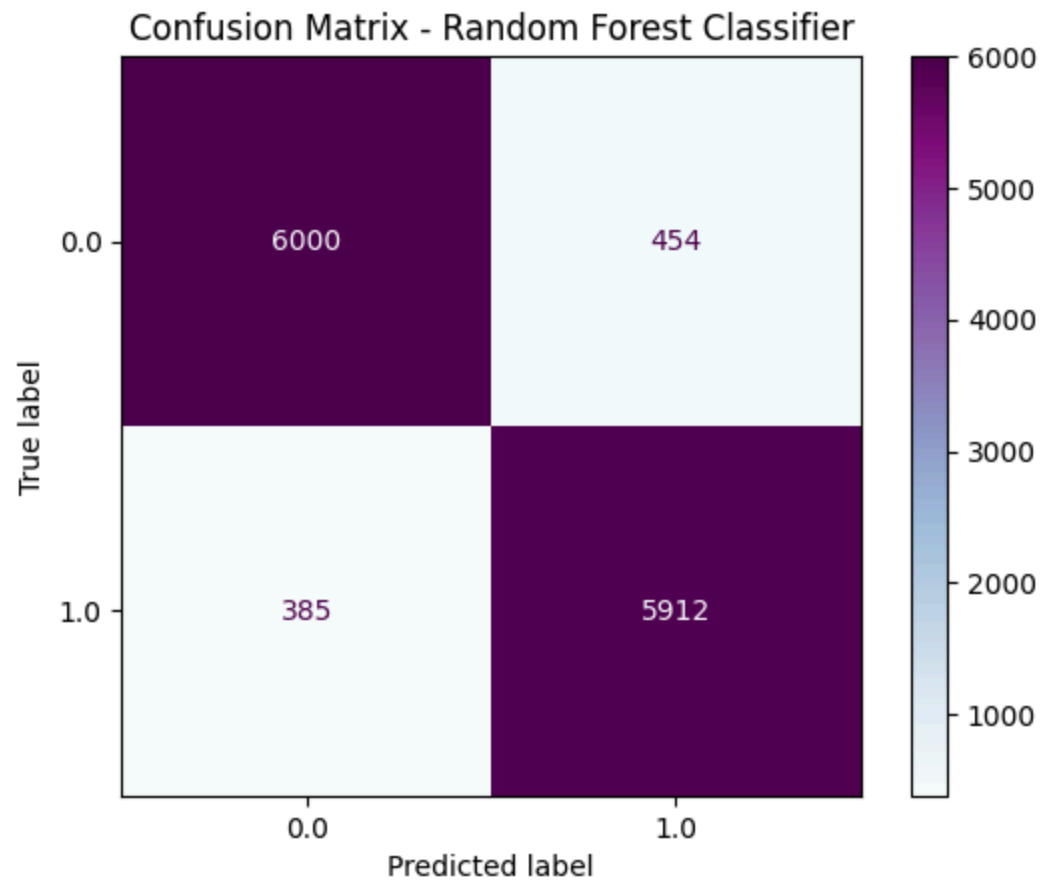
```
In [36]: grid_3 = GridSearchCV(model_3, params_3, cv = 5, scoring = 'f1')  
grid_3.fit(X_train, y_train)
```

```
print(grid_3.best_params_)  
print(grid_3.best_score_)
```

```
{'max_depth': 10, 'min_samples_split': 3, 'n_estimators': 200}  
0.9327160867332672
```

```
In [38]: model3 = RandomForestClassifier(random_state=12, class_weight='balanced', max_depth=10, min_samples_split=3, n_estimators=200)  
model3.fit(X_train, y_train)  
pred_y_3 = model3.predict(X_test)  
ConfusionMatrixDisplay.from_predictions(y_test, pred_y_3, cmap = 'BuPu')  
plt.title("Confusion Matrix - Random Forest Classifier")
```

```
Out[38]: Text(0.5, 1.0, 'Confusion Matrix - Random Forest Classifier')
```



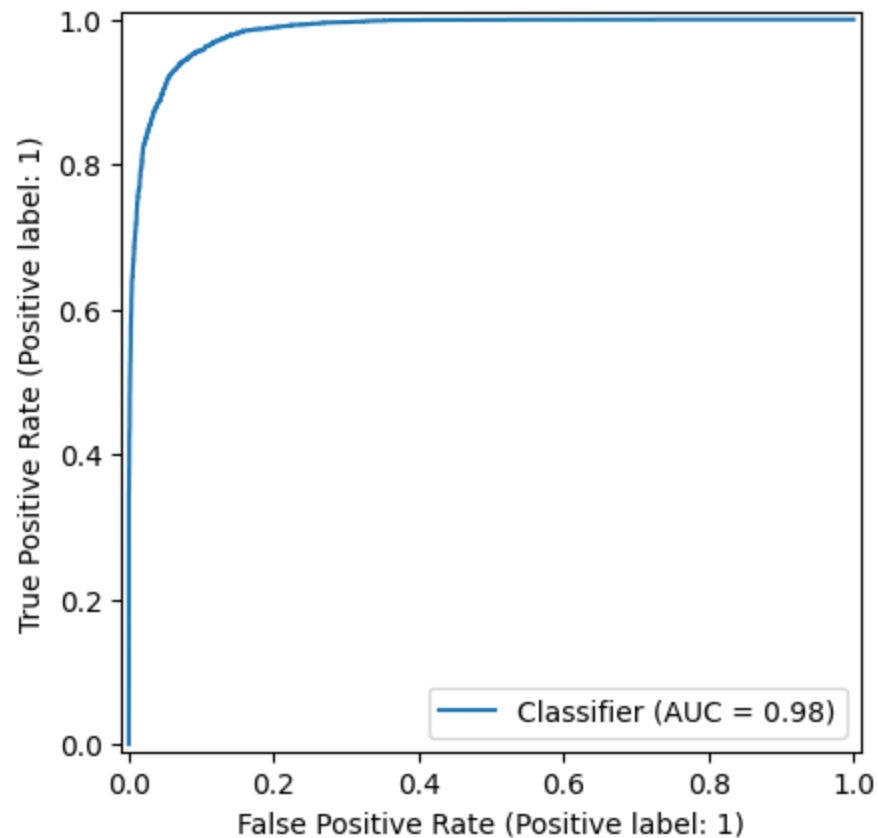
```
In [38]: print("Accuracy : ",round(accuracy_score(y_test, pred_y_3),2))
print("Precision : ",round(precision_score(y_test, pred_y_3),2))
print("Recall : ",round(recall_score(y_test, pred_y_3),2))
print("f1_Score : ",round(f1_score(y_test, pred_y_3),2))
```

```
Accuracy : 0.93
Precision : 0.93
Recall : 0.94
f1_Score : 0.93
```

```
In [39]: y_scores = model3.predict_proba(X_test)[: ,1]

RocCurveDisplay.from_predictions(y_test, y_scores)
```

```
Out[39]: <sklearn.metrics._plot.roc_curve.RocCurveDisplay at 0x2690d35d310>
```



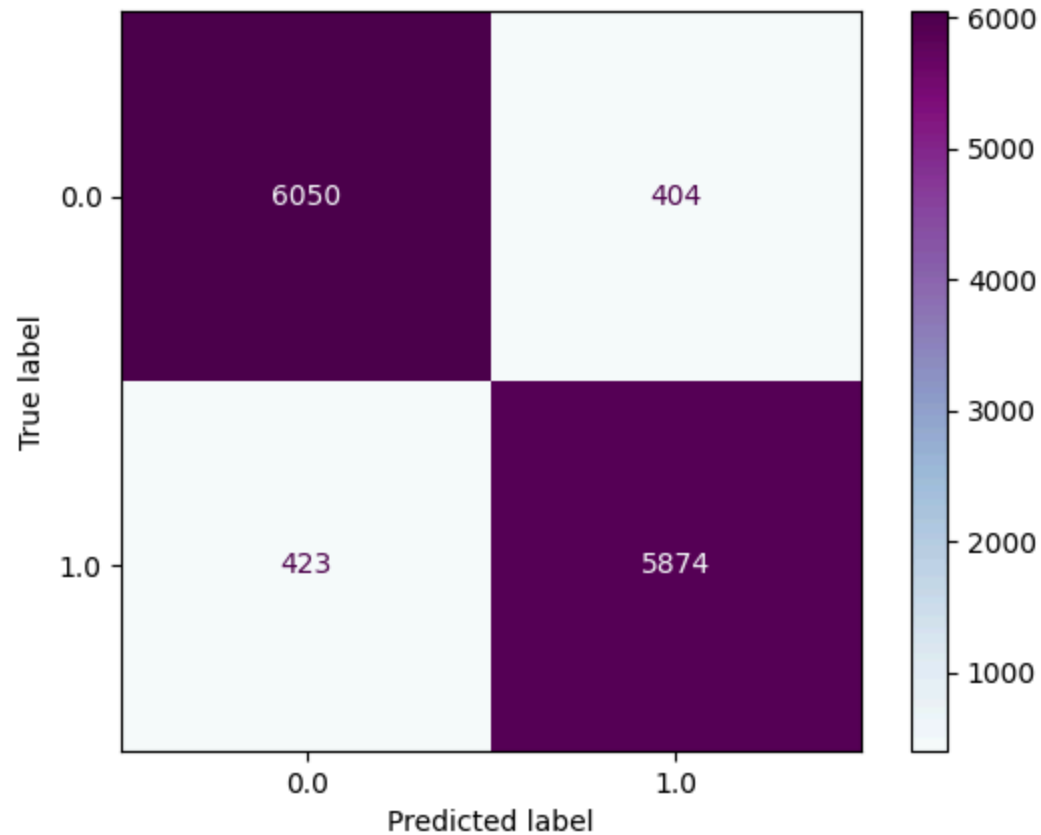
```
In [ ]: grid_4 = GridSearchCV(model_4, params_4, cv = 5, scoring='f1')
        grid_4.fit(X_train, y_train)

        print(grid_4.best_params_)
        print(grid_4.best_score_)
```

```
{'gamma': 3, 'learning_rate': 0.1, 'scale_pos_weight': 1}
0.9343608564851852
```

```
In [40]: model4 = XGBClassifier(gamma = 3, learning_rate = 0.1, scale_pos_weight = 1)
        model4.fit(X_train, y_train)
        pred_y_4 = model4.predict(X_test)
        ConfusionMatrixDisplay.from_predictions(y_test, pred_y_4, cmap = 'BuPu')
```

```
Out[40]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x2690d3c1ed0>
```



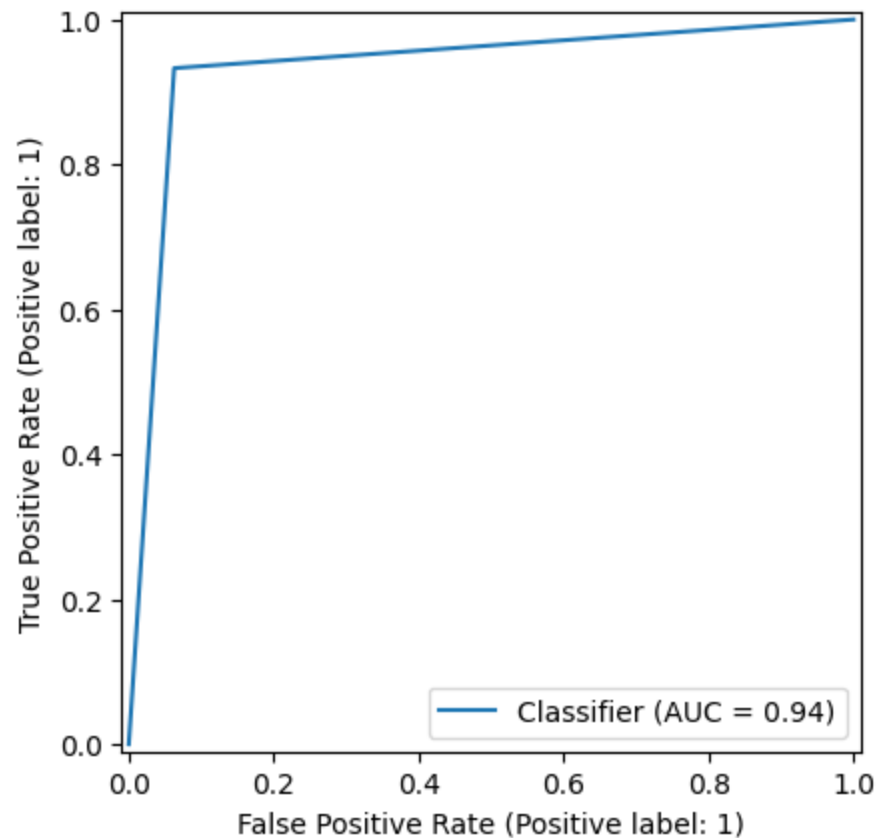
```
In [41]: print("Accuracy : ",round(accuracy_score(y_test, pred_y_4),2))
print("Precision : ",round(precision_score(y_test, pred_y_4),2))
print("Recall : ",round(recall_score(y_test, pred_y_4),2))
print("f1_Score : ",round(f1_score(y_test, pred_y_4),2))
```

```
Accuracy : 0.94
Precision : 0.94
Recall : 0.93
f1_Score : 0.93
```

```
In [42]: y_scores = model4.predict_proba(X_test)[: ,1]

RocCurveDisplay.from_predictions(y_test, pred_y_4)
```

```
Out[42]: <sklearn.metrics._plot.roc_curve.RocCurveDisplay at 0x2691567b650>
```



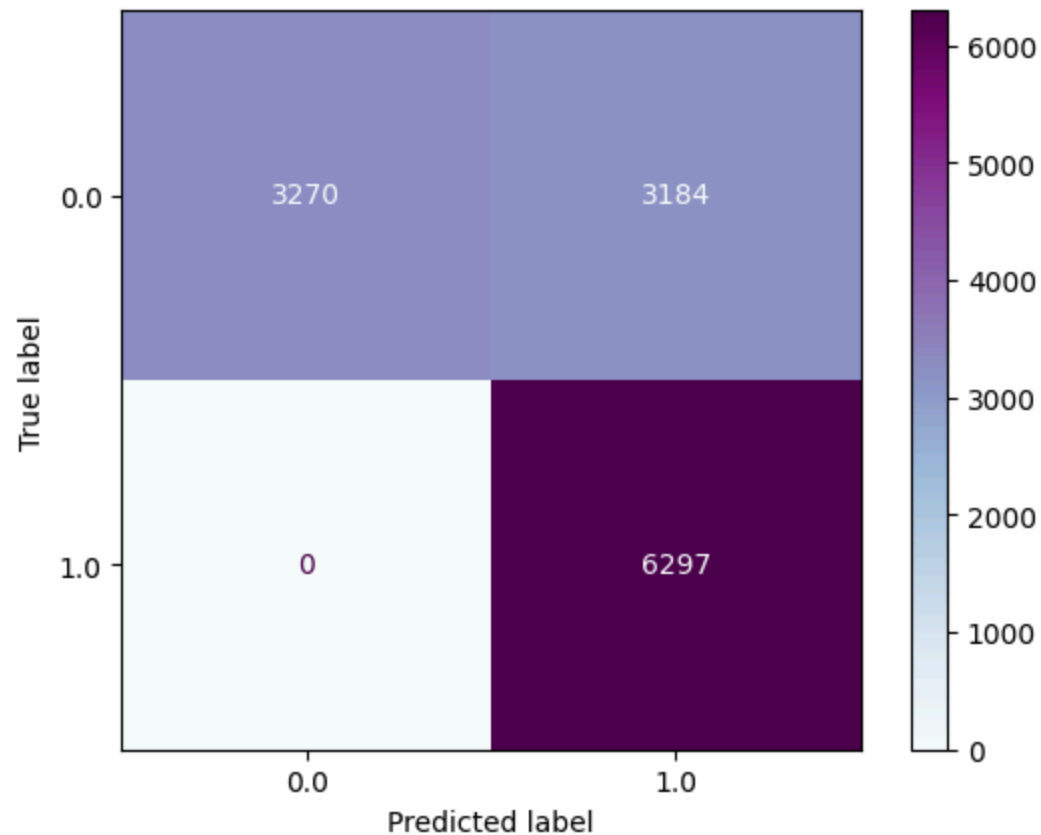
```
In [43]: grid_5 = GridSearchCV(model_5, params_5, cv = 5, scoring = 'f1')
grid_5.fit(X_train_scaled, y_train)

print(grid_5.best_params_)
print(grid_5.best_score_)
```

```
{'alpha': 0.001, 'hidden_layer_sizes': (16,), 'solver': 'adam'}
0.7948394888509815
```

```
In [44]: model5 = MLPClassifier(random_state=12, max_iter=500, alpha=0.001, hidden_layer_sizes=(16,), solver='adam')
model5.fit(X_train_scaled, y_train)
pred_y_5 = model5.predict(X_test_scaled)
ConfusionMatrixDisplay.from_predictions(y_test, pred_y_5, cmap = 'BuPu')
```

```
Out[44]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x269156f6590>
```



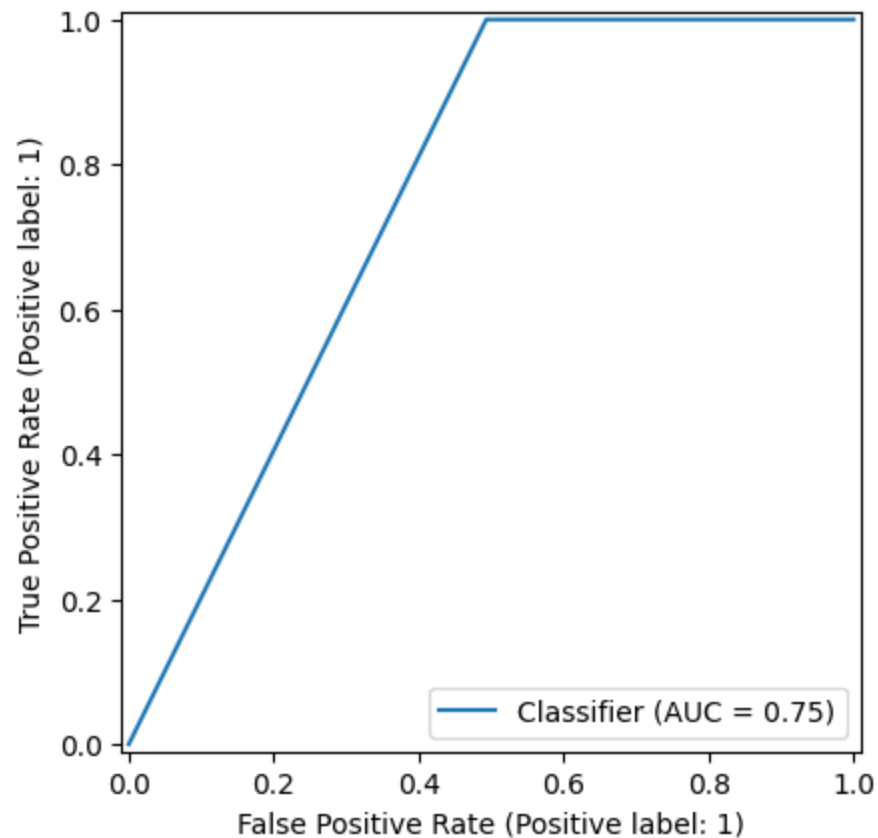
```
In [45]: print("Accuracy : ",round(accuracy_score(y_test, pred_y_5),2))
print("Precision : ",round(precision_score(y_test, pred_y_5),2))
print("Recall : ",round(recall_score(y_test, pred_y_5),2))
print("f1_Score : ",round(f1_score(y_test, pred_y_5),2))
```

```
Accuracy : 0.75
Precision : 0.66
Recall : 1.0
f1_Score : 0.8
```

```
In [46]: y_scores = model5.predict_proba(X_test_scaled)[: ,1]

RocCurveDisplay.from_predictions(y_test, pred_y_5)
```

```
Out[46]: <sklearn.metrics._plot.roc_curve.RocCurveDisplay at 0x2691574c410>
```

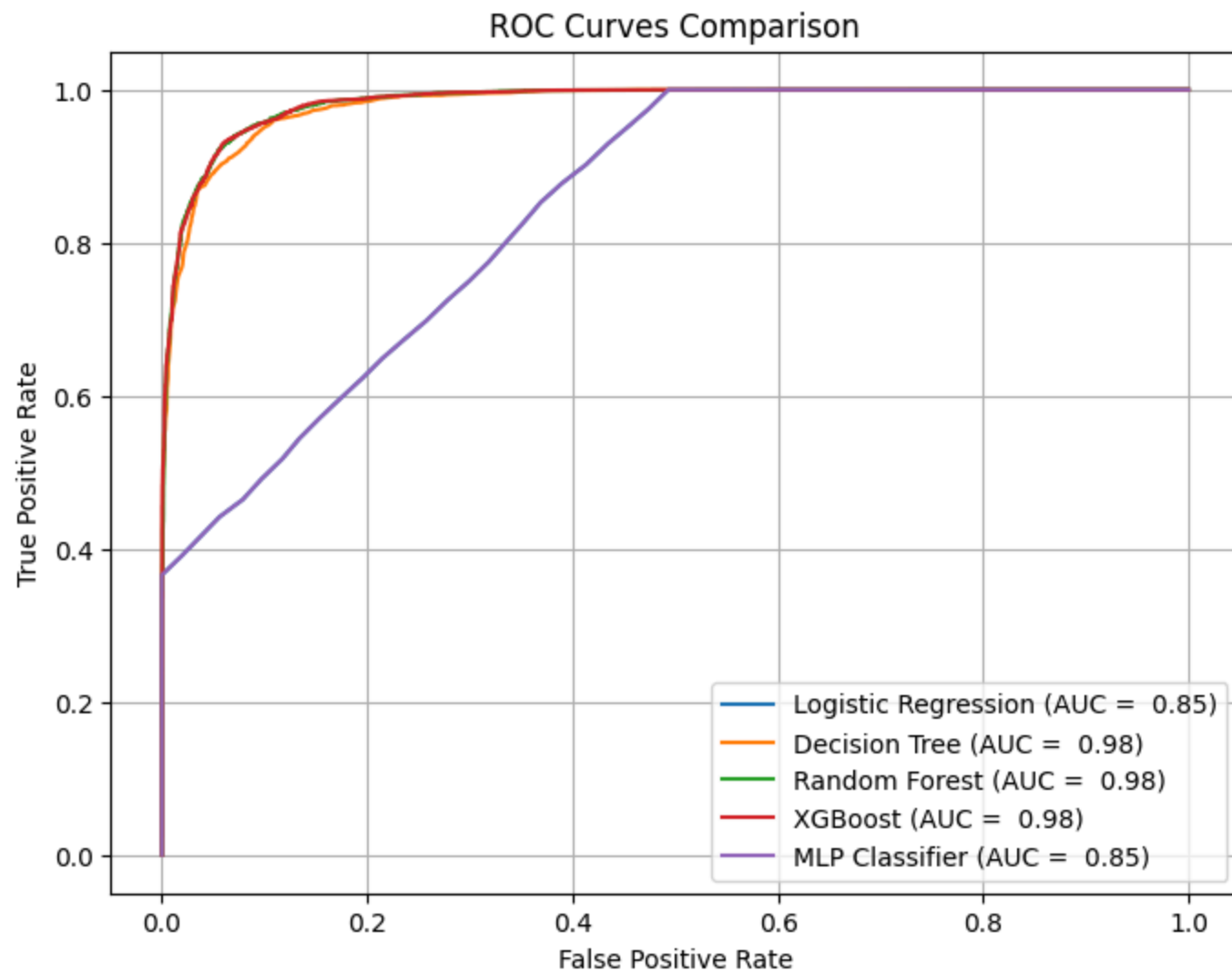


```
In [49]: plt.figure(figsize = (8,6))

models = [
    ("Logistic Regression", model1, X_test_scaled),
    ("Decision Tree", model2, X_test),
    ("Random Forest", model3, X_test),
    ("XGBoost", model4, X_test),
    ("MLP Classifier", model5, X_test_scaled)
]

for name, model, X in models:
    y_scores = model.predict_proba(X)[:,:1] #[:,1] means "take the probability of the positive class (1)" for every
    fpr, tpr, _ = roc_curve(y_test, y_scores)
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, label = f"{name} (AUC = {roc_auc: .2f})")

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curves Comparison")
plt.legend(loc = "lower right")
plt.grid(True)
plt.show()
```



According to the ROC curve, both the Decision Tree model and the Random Forest model give the same AUC (Area Under the Curve). However, based on the confusion matrix, we can conclude that the Random Forest Classifier is the best model between the two. Because, its confusion matrix shows fewer False Positives and False Negatives.